

Motor de Base de Datos Relacional

Documentación Técnica
Nombres
Jefferson David Rico Ruiz
Kevin Sebastian Correa Buritica
Kevin Felipe Vargas Farfan

1. Arquitectura General del Sistema

El motor se organiza en cinco capas con responsabilidad única. Cada capa expone una interfaz bien definida al nivel superior y desconoce los detalles internos de las demás.

Módulo	Archivo	Responsabilidad
REPL	parsing/repl.py	Interfaz CLI interactiva. Ciclo Read-Eval-Print. Comandos especiales: .help, .tables, .clear, exit/quit.
Engine	parsing/engine.py	Orquestador central. Mantiene self.tables y self.catalog. Reconstruye árboles B+ al iniciar.
Lexer	parsing/lexer.py	Tokenización con regex. Produce lista de Token normalizada (keywords en mayúsculas).
Parser	parsing/parser.py	Análisis sintáctico. Genera ASTNode por comando. Lanza SyntaxError ante gramática inválida.
BPlusTree	Tree/BPlusTree.py	Índice principal: una instancia por tabla. O(log n) en búsqueda, inserción y eliminación.
Catalog	Catalog/catalog.py	Metadatos de tablas y columnas. Se persiste en data/catalog.json tras cada CREATE/DROP.
DiskPersistence	Storage/disk_persistence.py	Acceso físico. Páginas de 4 096 bytes alineadas con el sistema operativo.

Flujo de una consulta SQL:

```
REPL → Engine.execute(sql) → Lexer.tokenize() → Parser.parse() → ASTNode
      → Engine.dispatch() → do_create / do_insert / do_select / do_update / do_delete /
do_drop
      → BPlusTree (operación) ; Catálogo actualizado en CREATE / DROP
```

2. Especificación del Lenguaje SQL Soportado

Comando	Sintaxis	Descripción
CREATE TABLE	CREATE TABLE t (col tipo [, ...])	Crea tabla. El primer campo declarado es la clave primaria implícita.
DROP TABLE	DROP TABLE t	Elimina la tabla, el árbol B+ en memoria y la entrada del catálogo.
INSERT INTO	INSERT INTO t VALUES (v1, v2, ...)	Inserta un registro. v1 es la clave primaria y debe ser único.
SELECT	SELECT * FROM t [WHERE col op val]	Sin WHERE: $O(n)$ recorre hojas enlazadas. Con WHERE sobre clave pk: $O(\log n)$.
UPDATE	UPDATE t SET col = val WHERE pk = clave	Localiza el registro por clave primaria y actualiza el campo indicado en SET.
DELETE FROM	DELETE FROM t WHERE pk = clave	Elimina el registro con la clave indicada y rebalancea el árbol B+ si hay déficit.

Tipos de datos soportados:

Tipo	Descripción	Ejemplo de valor
int	Entero de precisión arbitraria (Python int)	42
str	Cadena de texto entre comillas simples	'Ana'
float	Número decimal de punto flotante	3.14

Operadores permitidos en cláusula WHERE: = , != , > , < , >= , <=

Comandos especiales de la REPL:

Comando	Descripción
.help	Lista todos los comandos disponibles en la REPL.
.tables	Muestra las tablas registradas con su conteo de registros.
.clear	Limpia la consola (cls en Windows, clear en Unix/macOS).
exit / quit	Cierra el motor limpiamente.

3. Diseño del Árbol B+: Invariantes y Mantenimiento

Cada tabla es una instancia independiente de BPlusTree con order = 4 (máximo 4 claves por nodo). Jerarquía de clases: BPlusNode (nodo base: order, is_leaf, keys, children), BPlusLeafNode (hereda BPlusNode, agrega records y puntero next), BPlusTree (API pública).

Invariante	Descripción
Claves por nodo interno	Entre $\lceil t/2 \rceil - 1$ y $t-1$ claves. La raíz puede tener entre 1 y $t-1$ claves.
Claves por nodo hoja	Entre $\lceil t/2 \rceil$ y t registros. Todas las claves reales residen en nodos hoja.
Altura uniforme	Todos los caminos raíz→hoja tienen la misma longitud $h = O(\log_t n)$.
Separadores internos	Claves en nodos internos son copias de claves hoja; sirven solo para navegación.
Lista enlazada de hojas	Cada hoja mantiene puntero next a la siguiente → SELECT * recorre en $O(n)$.
Orden estricto de claves	Dentro de cada nodo, las claves están ordenadas estrictamente de menor a mayor.

Cómo se mantienen los invariantes:

Inserción — Se desciende hasta la hoja destino con insert_non_full(), dividiendo hijos llenos proactivamente (estrategia top-down). Al dividir una hoja la clave mediana se copia al padre (permanece en hoja derecha). Al dividir un nodo interno, la clave mediana sube al padre. Si la raíz se divide, se crea nueva raíz y el árbol crece en altura.

Eliminación — _delete() recursivo localiza la hoja. Si queda con déficit se llama _rebalance(): préstamo del hermano izquierdo o derecho si tiene claves de sobra; si ninguno presta, _merge() fusiona los nodos, actualiza los punteros next de la lista enlazada y elimina la clave separadora del padre. El proceso puede propagarse hasta la raíz.

Esquema visual (order = 4):



4. Formato de Persistencia y Política de Recuperación

Componente	Formato	Detalles
Catálogo de metadatos	JSON (data/catalog.json)	Estructura: { "tables": { "tabla": { "columns": { "name": "id", "type": "int", ... } } } }. Se sobrescribe tras cada CREATE TABLE y DROP TABLE.
Datos de registros	Páginas binarias (.db)	PAGE_SIZE = 4 096 bytes (alineado con el OS). Posición de página $n = n \times \text{PAGE_SIZE}$. Si el archivo es más corto, read_page() rellena con ceros (b'\x00').
Estado de integración	Solo catálogo es durable	El árbol B+ opera completamente en RAM. DiskPersistence existe como módulo autónomo pero no está integrado con el Engine en esta versión.

Política de recuperación al reinicio:

Al inicializarse, DatabaseEngine.__init__() ejecuta el siguiente protocolo:

1. Lee data/catalog.json mediante Catalog.load().
2. Para cada tabla registrada, reconstruye un BPlusTree vacío (order = 4) en memoria.
3. El esquema (columnas y tipos) queda disponible de inmediato.
4. Los registros DML no se restauran (volatilidad en esta versión).

Las operaciones DDL (CREATE / DROP) son durables entre sesiones. Las operaciones DML (INSERT / UPDATE / DELETE) son volátiles. El siguiente paso de evolución natural sería serializar las páginas del árbol B+ en disco tras cada escritura, activando la integración con DiskPersistence.

5. Análisis de Complejidad

Sea n = número de registros en la tabla, t = order = 4 y $h = O(\log_t n)$ la altura del árbol B+.

Operación / Comando SQL	Tiempo	Espacio	Observación
search(key) / SELECT WHERE pk	$O(\log n)$	$O(1)$	h descensos; $O(t)$ comparaciones por nodo.
insert() / INSERT INTO	$O(\log n)$	$O(\log n)$	Pila de recursión de profundidad h .
delete() / DELETE FROM	$O(\log n)$	$O(\log n)$	Hasta h rebalanceos propagados hacia la raíz.
update() / UPDATE	$O(\log n)$	$O(1)$	search() + modificación in-place del registro.
Recorrido de hojas / SELECT *	$O(n)$	$O(1)$	Sigue punteros next sin descender por el árbol.
CREATE TABLE / DROP TABLE	$O(1)$	$O(1)$	Árbol vacío + escritura del catálogo JSON.
Espacio total del árbol (n registros)	—	$O(n)$	Nodos internos $O(n/t)$; hojas almacenan $O(n)$ registros.

6. API de Módulos

6.1 Lexer — parsing/lexer.py

tokenize() → list[Token] Tipos de token: KEYWORD · IDENTIFIER · NUMBER · STRING · OPERATOR · COMMA · LPAREN · RPAREN · STAR

6.2 Parser — parsing/parser.py

Método	Descripción
parse()	Detecta el tipo de comando y delega a parse_select / insert / update / delete / create / drop.
parse_where()	Retorna diccionario {field, operator, value} para la cláusula WHERE.
parse_column_defs()	Retorna lista [{name, type}] para el comando CREATE TABLE.
consume(type, val)	Consume el token actual; lanza SyntaxError si tipo o valor no coinciden.

6.3 Engine — parsing/engine.py

Método	Descripción
execute(sql)	Tokeniza, parsea y despacha la consulta. Captura y muestra errores de sintaxis y ejecución.
dispatch(ast)	Enruta el ASTNode al método do_*() correcto según ast.type.
do_create(ast)	Crea BPlusTree(order=4) y registra la tabla en el catálogo.
do_drop(ast)	Elimina el árbol de self.tables y borra la entrada del catálogo.
do_insert(ast)	Inserta el registro; el primer valor del VALUES es la clave primaria.
do_select(ast)	Con WHERE sobre pk: O(log n). Sin WHERE: O(n) recorriendo hojas enlazadas.
do_update(ast)	Busca el registro por clave primaria y actualiza el campo indicado en SET.
do_delete(ast)	Elimina el registro con la clave del WHERE; el árbol se rebalancea si hay déficit.

6.4 BPlusTree — Tree/BPlusTree.py

Método	Descripción
insert(key, record)	Inserta clave y registro. Divide nodos si se desbordan. O(log n).
search(key) → any None	Retorna el registro asociado a la clave, o None si no existe. O(log n).
delete(key)	Elimina clave y registro; rebalancea por préstamo o merge si hay déficit. O(log n).
update(key, new_record)	Actualiza el registro asociado a la clave. Retorna True si tuvo éxito, False si no existe.
split_child_Tree_BPlus(p,i)	Divide hijo i: hojas copian clave mediana al padre; internos suben la clave al padre.
_rebalance(parent, index)	Intenta préstamo de hermano izq/der; si ambos tienen mínimo de claves, llama _merge().
_merge(parent, index)	Fusiona hijo izquierdo con derecho, actualiza lista enlazada de hojas y elimina separador.

6.5 Catálogo — Catalog/catalog.py

register_table(name, cols) · drop_table(name) · table_exists(name) → bool · get_columns(name) → list[{name,type}] · list_tables() → list[str] · describe(name) → str · show_tables() → str · load() / save()

6.6 DiskPersistence — Storage/disk_persistence.py

__init__(filename: str) · write_page(page_num: int, data: bytes) — escribe PAGE_SIZE bytes exactos, AssertionError si len(data) ≠ PAGE_SIZE · read_page(page_num: int) → bytes · total_pages() → int